

Region-based Object Detection

R-CNN and Fast R-CNN, then a first look at autoencoders

Sayan Chaki

Deep Learning for Computer Vision

Imagine team, LIRIS (UMR 5205), Université Lumière Lyon 2

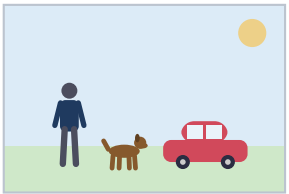
Today's journey

1. **The detection problem**
   IoU, AP, sliding windows
2. **R-CNN**
   regions + CNN features
3. **Fast R-CNN**
   share the computation
4. **Hands-on in Colab**
   both detectors on real data
5. **Autoencoders**
   simple, denoising, super-resolution

The detection problem

Three tasks, increasing difficulty

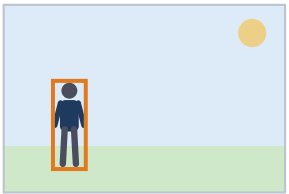
Classification



"person"

What is in the image?

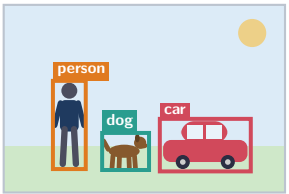
Localization



one object: class + box

What and *where*?

Detection



many objects: classes + boxes

What, where, and how many?

What exactly does a detector output?

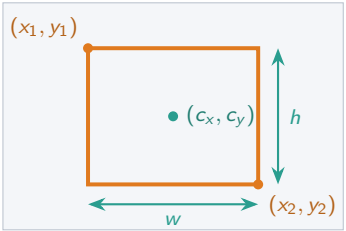


image coordinates (pixels)

A detection is a triple

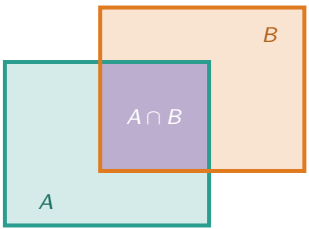
$$\Big(\underbrace{c}_{\text{class}},\ \underbrace{s}_{\text{score}\in[0,1]},\ \underbrace{b}_{\text{box}}\Big)$$

Two common box parametrisations:

- **corners** (x_1, y_1, x_2, y_2): easy to crop and to compute overlaps
- **centre-size** (c_x, c_y, w, h): natural for regression

The output is a set of variable size: 0, 1 or 50 objects. This alone rules out a plain "CNN $\rightarrow$ fixed vector" design.

Measuring overlap: Intersection over Union (IoU)

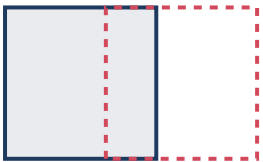


$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

$$= \frac{\text{overlap}}{\text{total area covered}}$$

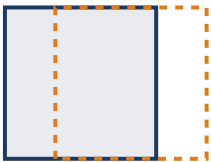
- 0: disjoint, 1: identical boxes
- scale invariant: only shapes matter, not pixels
- the universal "is this the same object?" test

IoU intuition: what does 0.5 look like?



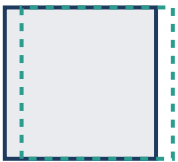
IoU $= 0.20$

poor



IoU $= 0.50$

PASCAL "correct" threshold



IoU $= 0.80$

good

Solid: ground truth. Dashed: prediction shifted horizontally. Even a box that looks "half wrong" scores only 0.5: IoU is a strict metric.

From detections to a score: precision, recall, AP

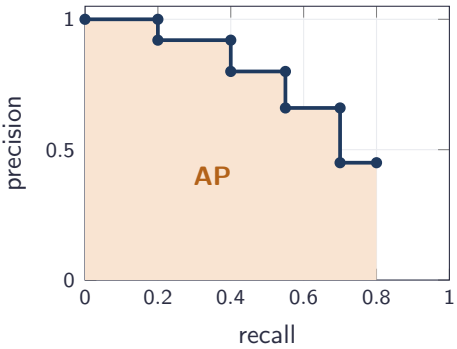
Matching rule (per class)

1. sort detections by score
2. a detection is a **TP** if IoU ≥ 0.5 with an *unmatched* ground truth
3. otherwise it is a **FP** (duplicates count as FP!)

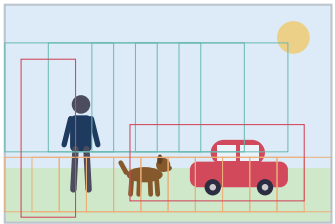
$$\text{precision} = \frac{TP}{TP+FP}, \qquad \text{recall} = \frac{TP}{\#\text{ground truth}}$$

AP = area under the precision/recall curve.

mAP = mean of AP over classes.



A naive detector: slide a classifier everywhere



Run a classifier on every window: every position (stride 8 px), scale ($\times 5$) and aspect ratio ($\times 3$). For a 500×375 image:

$$\frac{500}{8} \cdot \frac{375}{8} \cdot 5 \cdot 3 \approx 4 \times 10^4 \text{ windows}$$

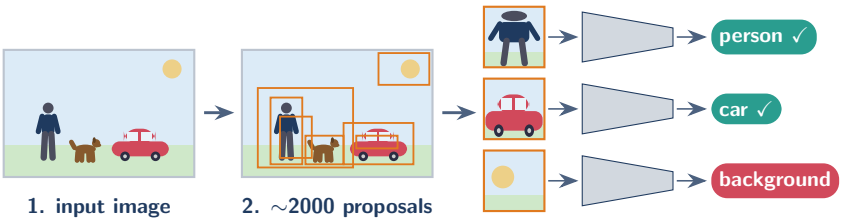
Fine for cheap features (HOG, DPM), hopeless with a deep CNN: one forward pass per window.

Key idea. look only where objects are likely: a few thousand *region proposals*.

R-CNN: Regions with CNN features

R-CNN at a glance

Girshick, Donahue, Darrell, Malik. *Rich feature hierarchies for accurate object detection and semantic segmentation*. CVPR 2014.



Detection = **where to look** (proposals) + **what is it** (CNN classifier) + **fix the box** (regression) + **remove duplicates** (NMS)

Step 1. Where to look: Selective Search

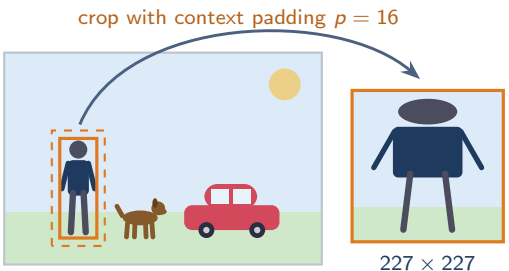
Uijlings, van de Sande, Gevers, Smeulders. *Selective Search for Object Recognition*. IJCV 2013.



- similarity = colour + texture + size + "fill"
- class agnostic, no learning, ∼2 s/image on CPU

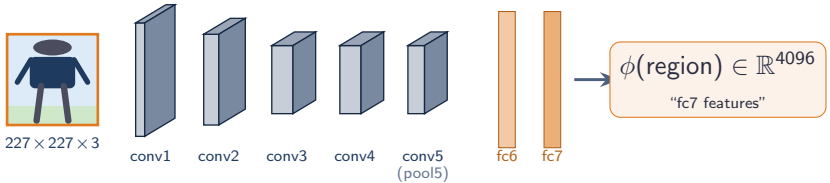
- goal: high recall with ∼ 2000 boxes
- precision does not matter here: the CNN filters later

Step 2. Warp every region to a fixed size



- the CNN (AlexNet) expects a fixed input size
- each proposal is enlarged by $p = 16$ pixels of context, then **anisotropically** scaled
- aspect ratio is *not* preserved: tall people become wide people
- surprisingly, this simple warp worked best among the options tested in the paper

Step 3. A CNN turns each region into a feature vector



1. Supervised pre-training

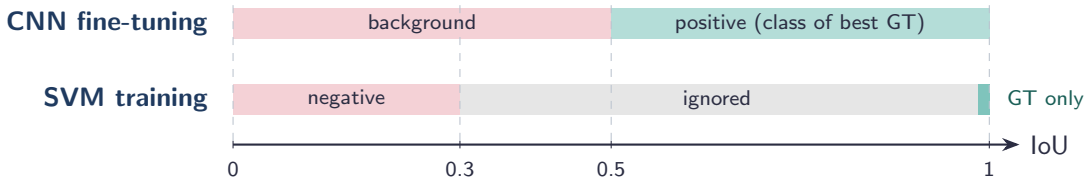
ImageNet classification, 1.2M images, 1000 classes. Learns generic visual features.

2. Domain-specific fine-tuning

Replace the last layer by $N+1$ outputs (N classes + background), train on warped proposals.

Transfer learning was key: detection datasets alone were too small.

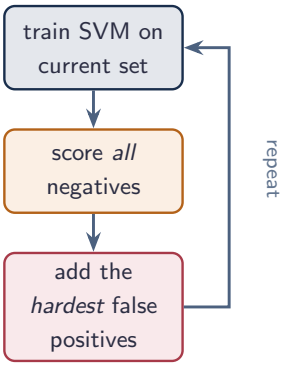
Labels for proposals: it all depends on IoU



Fine-tuning needs *lots* of data: a loose positive definition (≥ 0.5) multiplies the positives by $\sim 30\times$. Mini-batch of $128 = 32$ positives $+$ 96 background.

SVMs want *precise* positives: only ground-truth boxes. The grey zone is ambiguous, so it is simply ignored.

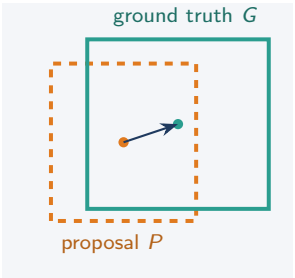
Step 4. One linear SVM per class + hard negative mining



$$s_c(\text{region}) = \mathbf{w}_c^\top \phi(\text{region}) + b_c$$

- negatives are millions; most are trivially easy (sky, grass)
- **hard negative mining**: keep only the ones the current model gets wrong
- why SVM and not the fine-tuned softmax? With the loose 0.5 positives, softmax scores were less precise (mAP 54.2% $\rightarrow$ 50.9% on VOC 2007).

Bounding-box regression: nudge the box



Predict the normalised offsets $\mathbf{t}$:

$$t_x = \frac{G_x - P_x}{P_w} \qquad t_y = \frac{G_y - P_y}{P_h}$$

$$t_w = \log \frac{G_w}{P_w} \qquad t_h = \log \frac{G_h}{P_h}$$

- divide by P_w, P_h: same correction for big and small boxes
- log: widths stay positive; "$\times 2$" and "$\div 2$" are symmetric
- R-CNN: ridge regression on pool5 features, trained on proposals with IoU > 0.6

Worked example: encoding and decoding a box

Encode (training target)

$P = (c_x{=}100,\ c_y{=}80,\ w{=}50,\ h{=}100)$
$G = (110,\ 75,\ 60,\ 90)$

$$t_x = (110 - 100)/50 = \mathbf{0.20}$$
$$t_y = (75 - 80)/100 = -\mathbf{0.05}$$
$$t_w = \log(60/50) = \mathbf{0.182}$$
$$t_h = \log(90/100) = -\mathbf{0.105}$$

Decode (test time)

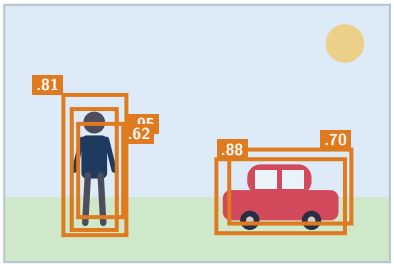
Given a prediction $\hat{\mathbf{t}}$, invert:

$$\hat{G}_x = P_w\,\hat{t}_x + P_x \qquad \hat{G}_w = P_w\,e^{\hat{t}_w}$$
$$\hat{G}_y = P_h\,\hat{t}_y + P_y \qquad \hat{G}_h = P_h\,e^{\hat{t}_h}$$

Check: $50 \cdot 0.2 + 100 = 110$ and $50 \cdot e^{0.182} = 60$.

In the notebook you will implement exactly these two functions: `encode_boxes` and `decode_boxes`.

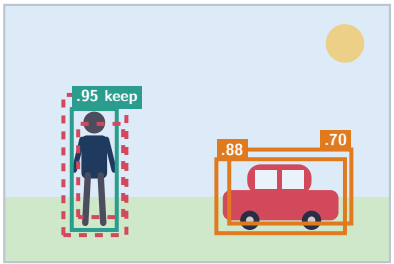
Removing duplicates: Non-Maximum Suppression (NMS)



Greedy NMS (per class), threshold τ (e.g. 0.3 to 0.5):

1. sort boxes by score

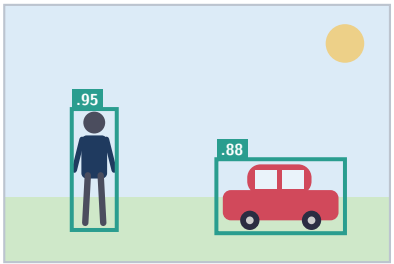
Removing duplicates: Non-Maximum Suppression (NMS)



Greedy NMS (per class), threshold τ (e.g. 0.3 to 0.5):

1. sort boxes by score
2. take the best box, keep it
3. delete every remaining box with IoU $> \tau$ with it

Removing duplicates: Non-Maximum Suppression (NMS)

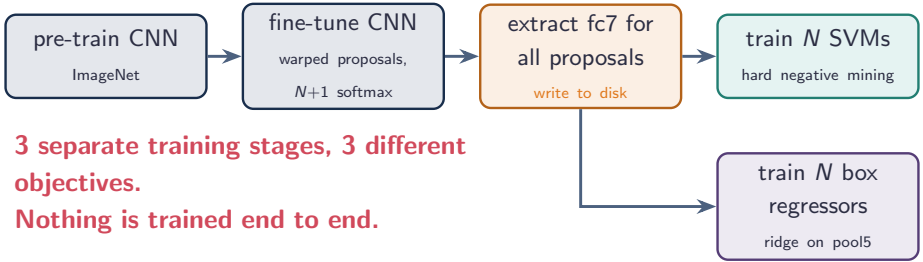


Greedy NMS (per class), threshold τ (e.g. 0.3 to 0.5):

1. sort boxes by score
2. take the best box, keep it
3. delete every remaining box with IoU $> \tau$ with it
4. repeat with what is left

One box per object. In PyTorch: `torchvision.ops.nms(boxes, scores, 0.3)`

R-CNN: the full recipe in one picture



3 separate training stages, 3 different objectives.
Nothing is trained end to end.

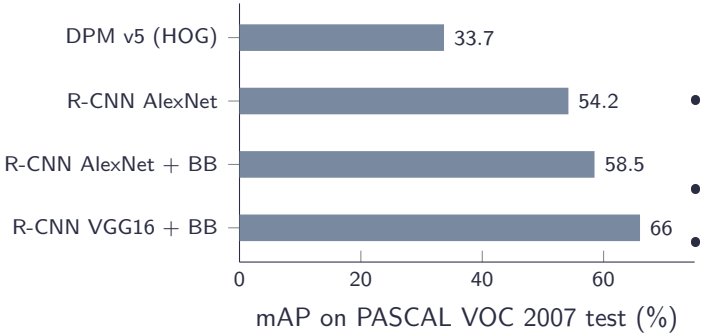
Test time, per image

$\sim$2000 proposals $\rightarrow$ 2000 CNN forward passes $\rightarrow$ SVM scores $\rightarrow$ box regression $\rightarrow$ NMS per class

Cost (VGG16 backbone)

training $\approx$ 84 h, features occupy hundreds of GB, test $\approx$ 47 s/image on a GPU

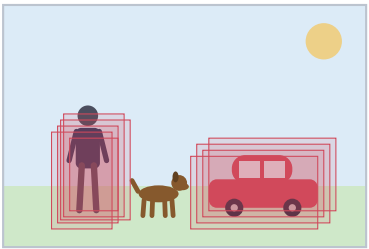
Results: a jump of more than 20 mAP points



- learned features beat hand-crafted HOG by a wide margin
- box regression: +4 points for free
- deeper backbone (VGG16): +7.5 points

Numbers from Girshick et al. 2014 (Tables 2 and 3).

What is wrong with R-CNN?



overlapping proposals: the same pixels go through the CNN again and again

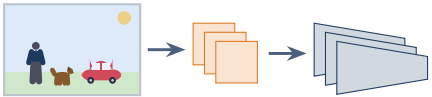
1. **Slow at test time.** 2000 independent forward passes, mostly on overlapping pixels.
2. **Multi-stage training.** CNN, SVMs and regressors are trained separately with different losses; the SVMs never improve the features.
3. **Expensive in space.** Features for every proposal of every image are cached on disk.

Key idea. run the CNN *once* per image and reuse its feature map for all regions.

Fast R-CNN: share the computation

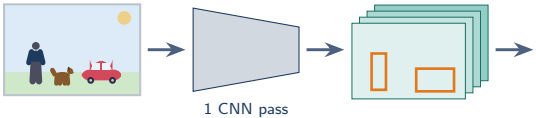
The central trick: crop the features, not the pixels

R-CNN: crop, then CNN, 2000 times



2000 CNN passes, mostly on the same pixels

Fast R-CNN: CNN once, then crop the feature map



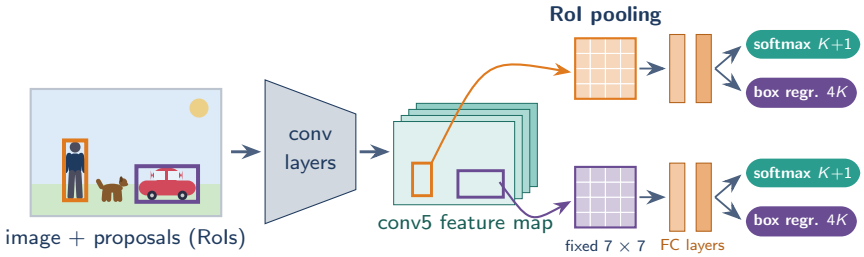
1 CNN pass

2000 cheap crops of the feature map

Convolutions are local and translation equivariant: the features of a region are (approximately) a sub-window of the features of the whole image. The idea first appeared in **SPPnet** (He et al., ECCV 2014); Fast R-CNN makes it trainable end to end.

Fast R-CNN architecture

Girshick. *Fast R-CNN*. ICCV 2015.

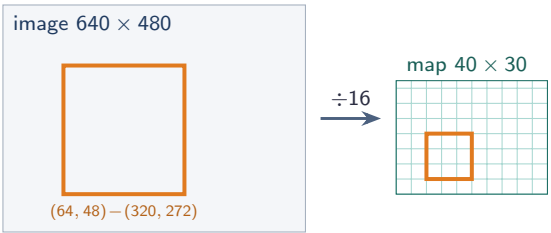


1. whole image through the CNN *once*

2. each RoI pooled to a fixed 7×7 grid

3. two sibling heads, *one* network, *one* loss

Projecting a proposal onto the feature map



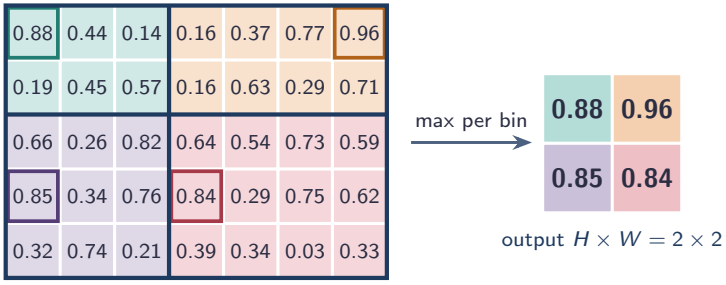
- VGG16 conv5 has 4 poolings $\Rightarrow$ **stride** 16
- a box (x_1, y_1, x_2, y_2) in the image becomes

$$\left(\left\lfloor \frac{x_1}{16} \right\rfloor, \left\lfloor \frac{y_1}{16} \right\rfloor, \left\lfloor \frac{x_2}{16} \right\rfloor, \left\lfloor \frac{y_2}{16} \right\rfloor\right)$$

- here: $(64, 48, 320, 272) \mapsto (4, 3, 20, 17)$, a 16×14 window of cells
- in PyTorch this is the `spatial_scale=1/16` argument

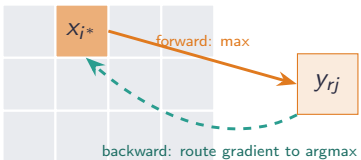
RoI pooling: any window $\rightarrow$ a fixed grid

projected RoI: 5×7 cells (one channel), split into 2×2 bins



Split the $h \times w$ window into an $H \times W$ grid of bins of size $\approx \frac{h}{H} \times \frac{w}{W}$, take the **max** in each bin, **independently per channel**. With VGG16, $H{=}W{=}7$: every RoI becomes $512 \times 7 \times 7$, exactly what `fc6` expects.

RoI pooling is differentiable



Like max pooling, the gradient flows only through the winning cell $i^*(r,j)$:

$$\frac{\partial L}{\partial x_i} = \sum_r \sum_j \left[i = i^*(r,j) \right] \frac{\partial L}{\partial y_{rj}}$$

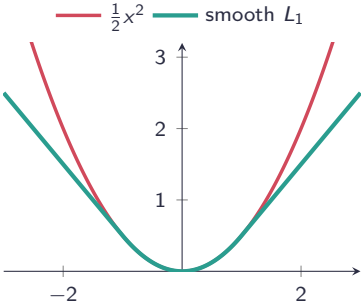
- a cell can win in *several* RoIs: gradients add up
- so the whole network, conv layers included, learns from the detection loss

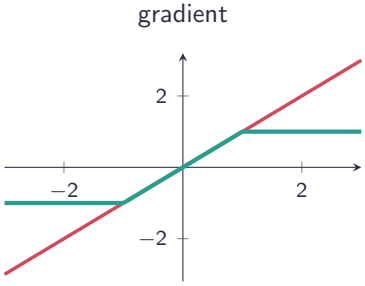
One network, one multi-task loss

$$L(p, u, t^u, v) = \underbrace{-\log p_u}_{\text{classification}} + \lambda \underbrace{[u \geq 1]}_{\text{not background}} \underbrace{\sum_{i \in \{x,y,w,h\}} \text{smooth}_{L_1}(t_i^u - v_i)}_{\text{localisation}}$$

- $p = (p_0, \ldots, p_K)$: softmax over K classes + background ($u = 0$)
- u: true class of the RoI, v: its regression target (same $\mathbf{t}$ as R-CNN)
- t^u: the box predicted *for class* u (class-specific regression, $4K$ outputs)
- Iverson bracket $[u \geq 1]$: a background RoI has no box to regress
- $\lambda = 1$, with targets normalised to zero mean, unit variance
- the SVMs disappear: the softmax is trained jointly and works as well

Why smooth L_1 and not L_2?





$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5\,x^2 & |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

Quadratic near 0 (smooth, precise), linear far away: the gradient is bounded, so badly matched RoIs cannot blow up training.

Efficient training: hierarchical sampling

fg IoU ≥ 0.5 (25%)

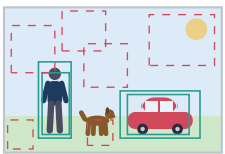


image 1: 64 RoIs

bg IoU $\in [0.1, 0.5)$

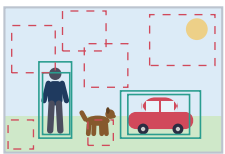
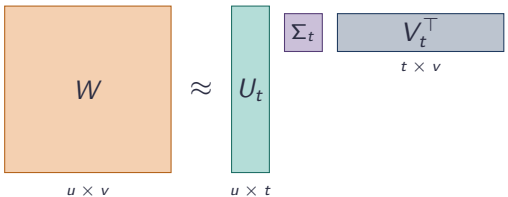


image 2: 64 RoIs

Mini-batch = N=2 images $\times$ 64 RoIs = 128 RoIs.

- RoIs from the *same* image share the conv computation: about 64$\times$ cheaper than 128 RoIs from 128 images
- 25% foreground, 75% background
- background starts at IoU 0.1: a crude form of hard example mining
- data augmentation: horizontal flip only
- VGG16: `conv1`–`conv2` frozen, everything above fine-tuned

Test-time speed-up: truncated SVD for the FC layers



parameters: $uv \longrightarrow t(u+v)$

- at test time there are ∼2000 RoIs, so nearly half the time is spent in `fc6` and `fc7`
- replace one FC layer by **two** smaller ones: $\Sigma_t V_t^\top$ (no bias) then U_t (with bias)
- keeping the top singular values (t=1024 for `fc6`, t=256 for `fc7`) cuts detection time by more than 30% for a 0.3 mAP drop

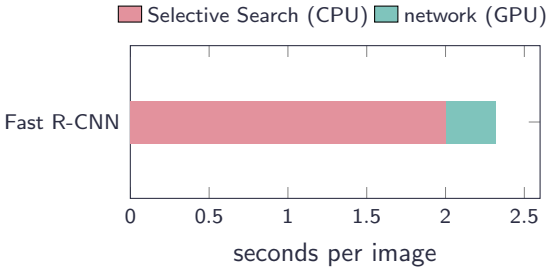
R-CNN vs SPPnet vs Fast R-CNN

VGG16, PASCAL VOC 2007	R-CNN	SPPnet	Fast R-CNN
training time (h)	84	25	**9.5**
training speed-up	1×	3.4×	**8.8**×
test time / image (s)	47.0	2.3	**0.32**
test speed-up	1×	20×	**146**×
with truncated SVD			0.22 s, 213×
mAP (%)	66.0	63.1	**66.9**
end-to-end training	✗	✗	✓
feature cache on disk	yes	yes	**no**

Girshick, Fast R-CNN, ICCV 2015, Table 4. Times exclude the proposal step.

Key idea. faster *and* more accurate: sharing computation also allowed training the conv layers end to end.

The new bottleneck and what came next



Proposals now cost $\sim 6\times$ more than the network itself.

2014 R-CNN
proposals + CNN + SVM

2014 SPPnet
shared conv features

2015 Fast R-CNN
RoI pooling, end to end

2015 Faster R-CNN
learned proposals (RPN)

2017 Mask R-CNN
RoIAlign + masks

Faster R-CNN predicts proposals with a small network on the *same* feature map; **RoIAlign** replaces the rounding of RoI pooling by bilinear interpolation.

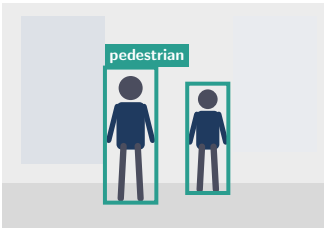
Cheat sheet

	R-CNN (2014)	Fast R-CNN (2015)
proposals	Selective Search, $\sim$2k	Selective Search, $\sim$2k
CNN input	each warped region	the whole image, once
region features	fc7 of the crop	RoI pooling on conv5, then fc6, fc7
classifier	N linear SVMs	softmax over $K+1$
box refinement	separate ridge regressors	sibling FC layer, smooth L_1
training	3 stages, cached features	single stage, multi-task loss
duplicates	NMS	NMS

Three questions you should be able to answer: *Why warp? Why does RoI pooling make the CNN run once? Why is there an Iverson bracket in the loss?*

Hands-on: both detectors in Colab

The lab: pedestrians in the Penn-Fudan dataset



170 street images, 345 annotated pedestrians, 1 class.
Small enough to train both detectors in minutes on a free T4.

Notebook `01_rcnn_fast_rcnn_pennfudan.ipynb`

1. load data, draw boxes, implement **IoU**
2. **Selective Search**: proposals and their recall
3. **R-CNN**: warped crops → AlexNet fc7 → linear SVM with hard negatives → box ridge regression → NMS
4. **Fast R-CNN**: VGG16 conv5 → `roi_pool` → two heads, trained end to end
5. compare **AP@0.5** and **seconds per image**

The heart of the Fast R-CNN code

```
class FastRCNN(nn.Module):
    def __init__(self, num_classes=1):
        super().__init__()
        vgg = torchvision.models.vgg16(weights="DEFAULT")
        self.backbone = vgg.features[:-1]    # conv5_3, stride
             16
        self.head = nn.Sequential(*list(vgg.classifier)[:-1])
             # fc6, fc7
        self.cls = nn.Linear(4096, num_classes + 1)
        self.reg = nn.Linear(4096, 4 * num_classes)

    def forward(self, images, rois):
        fmap = self.backbone(images)          # ONE pass per
             image
        pooled = roi_pool(fmap, rois, output_size=(7, 7),
                          spatial_scale=1 / 16)  # (R, 512, 7,
                               7)
        h = self.head(pooled.flatten(1))      # (R, 4096)
        return self.cls(h), self.reg(h)
```

- drop the last max-pool of VGG: `roi_pool` replaces it
- `rois` is a $(R, 5)$ tensor: `[batch_index, x1, y1, x2, y2]` in *image* pixels
- `spatial_scale` does the $\div 16$ projection for you
- the loss: `cross_entropy` + `smooth_l1_loss` on foreground RoIs only

What to look at when you run the notebook

Proposal recall

Plot recall@IoU 0.5 against the number of Selective Search boxes. How many proposals do you need before recall saturates?

Speed

Time one test image for each detector, *excluding* proposals. Where does R-CNN spend its time?

Accuracy

Compare AP@0.5, with and without box regression. Which errors does regression fix?

Failure cases

Look at false positives and missed pedestrians: occlusion? small people? groups merged by NMS?

Keep a small table of your numbers: we will compare across the room.

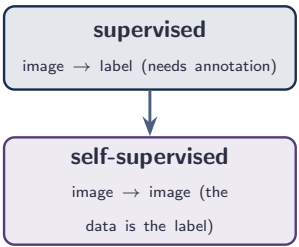
Autoencoders: learning by reconstructing

From labels to no labels

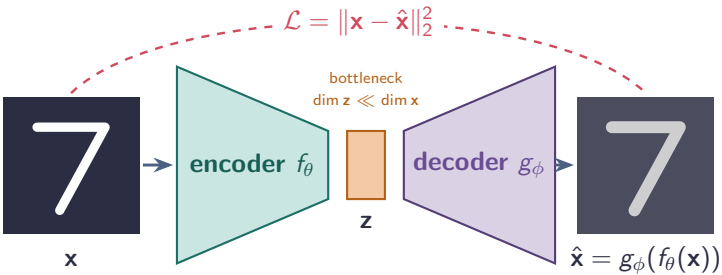
R-CNN worked because a CNN pre-trained on 1.2M *labelled* images gave rich features.

Question: can a network learn useful features with no labels at all?

Autoencoder answer: ask the network to *reproduce its input* through a narrow bottleneck. To succeed, it must discover the structure of the data.



Anatomy of an autoencoder



Encoder: compress the input into a code

Code z: the learned representation

Decoder: rebuild the input from the code

Why the bottleneck matters

The identity trap

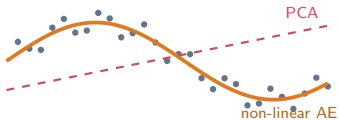
If $\dim \mathbf{z} \geq \dim \mathbf{x}$, the network can learn $\hat{\mathbf{x}} = \mathbf{x}$ without understanding anything. Zero loss, useless code.

Undercomplete autoencoder

A small $\mathbf{z}$ forces the model to keep only what explains most of the data and drop the rest.

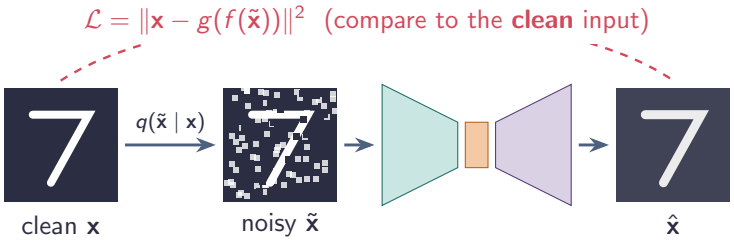
Link with PCA

A *linear* autoencoder trained with MSE learns the same subspace as PCA with $\dim \mathbf{z}$ components. Non-linear layers allow a *curved* subspace: a manifold.



Denoising autoencoder: reconstruct the clean input

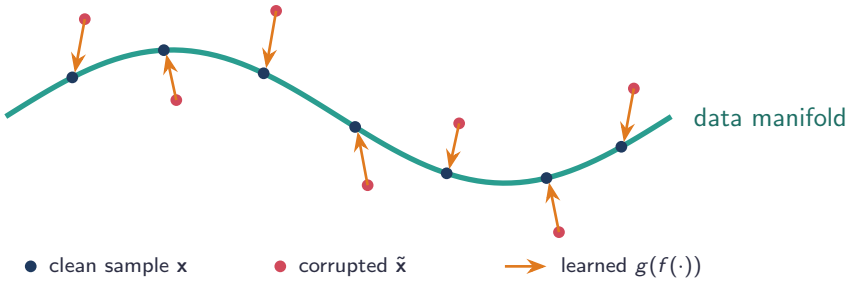
Vincent, Larochelle, Bengio, Manzagol. *Extracting and composing robust features with denoising autoencoders.* ICML 2008.



Corruptions q: Gaussian noise, salt-and-pepper, random masking of pixels. Copying is no longer a solution, even without a tight bottleneck.

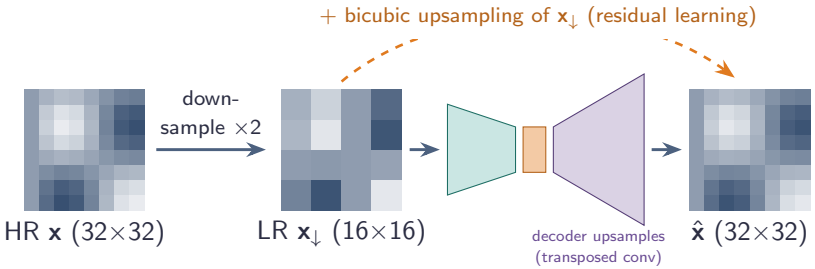
Intuition: the network learns to push points that fell *off* the data manifold back *onto* it.

The manifold picture of denoising



Natural images occupy a tiny, structured part of pixel space. A denoising autoencoder learns a map that sends nearby noisy points back to it: it has learned what natural images look like.

Super-resolution autoencoder: the input is a degraded image



Same recipe, different corruption: the "noise" is the loss of resolution. With the residual connection the network only learns the missing *details*.

Measure with the **PSNR** (pixels in $[0, 1]$):

$$\mathrm{PSNR} = 10 \log_{10} \frac{1}{\mathrm{MSE}(\mathbf{x}, \hat{\mathbf{x}})} \quad \mathrm{dB}$$

and always compare against the bicubic baseline.

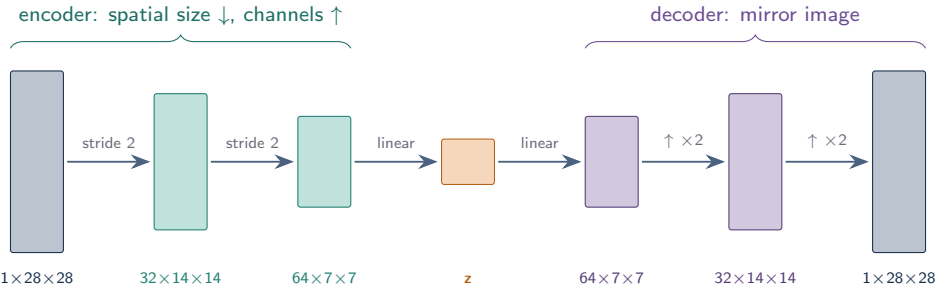
One framework, three tasks

	input to the network	target	what is learned
Autoencoder	$\mathbf{x}$	$\mathbf{x}$	compact code $\mathbf{z}$
Denoising AE	$\mathbf{x} + \text{noise}$	$\mathbf{x}$	robust features, denoiser
Super-resolution AE	$\text{down}(\mathbf{x})$	$\mathbf{x}$	missing high frequencies

```
loss = mse(model(corrupt(x)), x)
```

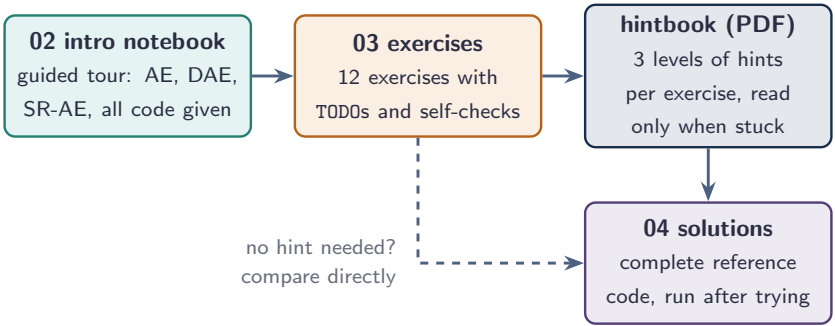
In the notebooks, the *same* training loop serves all three models: only `corrupt` changes.

A convolutional autoencoder, shape by shape



- stride-2 convolutions downsample, `ConvTranspose2d` upsamples
- `sigmoid` at the end since pixels live in $[0, 1]$
- always print the shapes: most bugs are off-by-one sizes
- `ConvTranspose2d(k=3, s=2, p=1, output_padding=1)` doubles the size exactly

Your autoencoder material



In Colab: *File* → *Upload notebook*, then *Runtime* → *Change runtime type* → *T4 GPU*.

Take-home messages

1. **Detection** = where to look + what it is + fix the box + remove duplicates. IoU and NMS are everywhere.
2. **R-CNN** showed that CNN features, pre-trained on ImageNet and fine-tuned, transform detection accuracy.
3. **Fast R-CNN** runs the CNN once and crops the *feature map* with RoI pooling: $\sim$150$\times$ faster at test time and trainable end to end with one multi-task loss.
4. **Autoencoders** learn without labels by reconstructing their input through a bottleneck.
5. **Denoising** and **super-resolution** are the same idea with a different corruption: `model(corrupt(x))` $\approx$ `x`.

References

- R. Girshick, J. Donahue, T. Darrell, J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. *CVPR*, 2014.
- K. He, X. Zhang, S. Ren, J. Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. *ECCV*, 2014.
- R. Girshick. Fast R-CNN. *ICCV*, 2015.
- S. Ren, K. He, R. Girshick, J. Sun. Faster R-CNN: towards real-time object detection with region proposal networks. *NeurIPS*, 2015.
- K. He, G. Gkioxari, P. Dollár, R. Girshick. Mask R-CNN. *ICCV*, 2017.
- J. Uijlings, K. van de Sande, T. Gevers, A. Smeulders. Selective search for object recognition. *IJCV*, 2013.
- M. Everingham et al. The PASCAL Visual Object Classes (VOC) challenge. *IJCV*, 2010.
- G. Hinton, R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 2006.
- P. Vincent, H. Larochelle, Y. Bengio, P.-A. Manzagol. Extracting and composing robust features with denoising autoencoders. *ICML*, 2008.
- C. Dong, C. C. Loy, K. He, X. Tang. Image super-resolution using deep convolutional networks. *TPAMI*, 2016.
- L. Wang, J. Shi, G. Song, I. Shen. Object detection combining recognition and segmentation. *ACCV*, 2007. (Penn-Fudan)